Predicting Repeat Orders: Building an End-to-End ML Pipeline with XGBoost & SHAP Interpretability

Nonchalant Team

Smart Predictive Analytic Research Competition
by Ciputra University

Business Objective

SPARC 2026 is a competition focused on leveraging data to support operational decision-making through a machine learning approach. The primary objective of the case studies in this competition is to help a company prioritize repeat order follow-up based on available data patterns and characteristics.

Dataset Overview

- Large-Scale Data Volume: The dataset provides a robust foundation for analysis with a total of 319,978 rows and 28 distinct features.
- Diverse Data Composition: The features are a mix of 5 numerical variables (such as OTR, Age, and Installments) and 23 categorical variables (such as Occupation, Religion, and Dealer Info).
- Significant Data Gaps: A quality assessment revealed a high volume of 576,759 missing values across various columns, indicating the need for thorough data cleaning.

Category	Information
Total Rows	319,978
Total Columns	28
Numerical Features	5
Categorical Features	23
Total Missing Values	576,759

End-To-End Workflow

01

Setup & Library Import

02

Data Exploration & Initial Visualization

03

Data Preprocessing Pipeline

04

Modeling with XGBoost

End-To-End Workflow

05

Model Evaluation

06

Model Explainability (SHAP Analysis)

07

Final Deliverables & Business Insights

IMPORT LIBRARY & SETUP

- Data handling & visualization: pandas, numpy, matplotlib, seaborn → load, clean, process tabular data and create clear statistical plots (scatter, box, heatmap).
- Machine learning pipeline & evaluation: scikit-learn modules + xgboost → build full pipeline (impute, scale, encode, split, cross-validate), train XGBoost model, and measure performance (F1, ROC-AUC, classification report).
- Explainability & utilities: shap, re, warnings → interpret model predictions (feature importance), clean text patterns with regex, and suppress annoying warnings for cleaner notebook output.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import shap
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, f1_score, roc_auc_score
from xgboost import XGBClassifier
```

Load Dataset

LOAD DATASET

```
url = "https://raw.githubusercontent.com/micelll/SPARC-2026/main/SPARC_dataset.csv"

df = pd.read_csv(url, low_memory=False)
df.columns = df.columns.str.strip().str.lower()

print({df.shape[0]} & {df.shape[1]})
```

Feature Engineering & Target Creation

- Target Definition (is_repeat) Count transactions per customer → >1 transaction = Repeat Buyer (1), otherwise = One-time Buyer (0)
- Currency & Numeric Cleaning Regex function + pd.to_numeric cleans currency formats (Rp, dots, commas) in DP, installment, OTR, and age columns
- Engineered Ratios + Class Distribution Created dp_ratio & cicilan_ratio → target distribution: ~72% one-time buyers, ~28% repeat buyers (class imbalance)

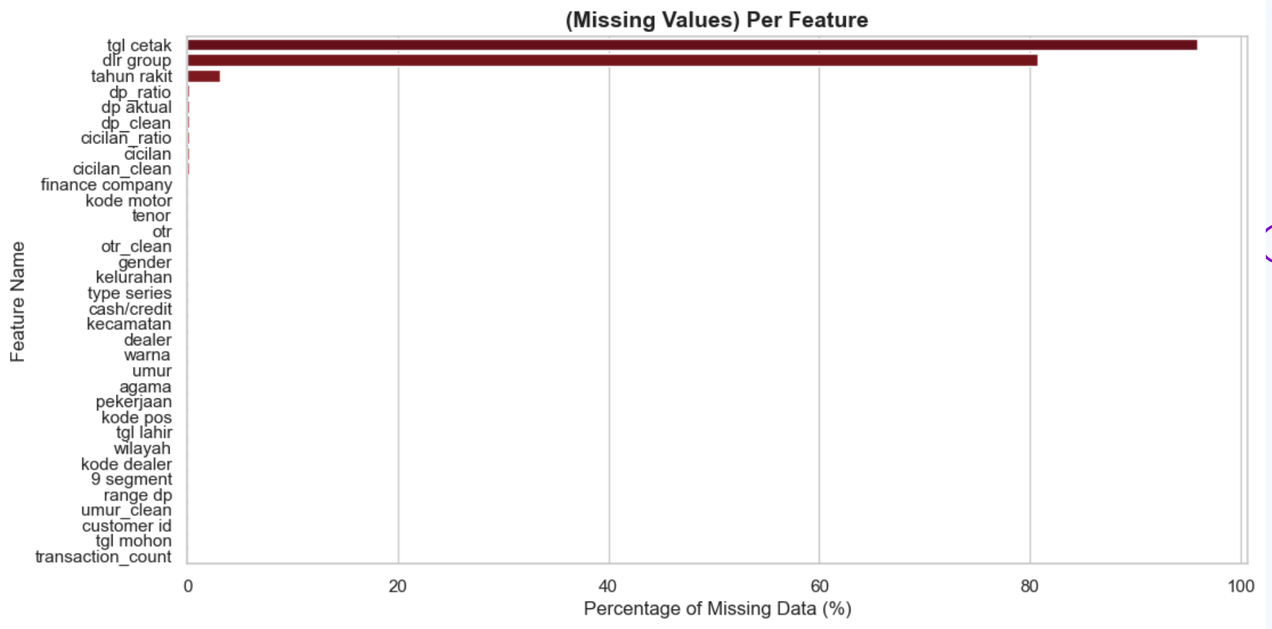
```
customer_count = df['customer id'].value_counts()
df['transaction_count'] = df['customer id'].map(customer_count)
df['is_repeat'] = (df['transaction_count'] > 1).astype(int)

# 2. Cleaning Currency Function
def clean_currency(val):
    if pd.isna(val): return np.nan
    numbers = re.findall(r"\d+", str(val))
    return float(numbers[-1]) if numbers else np.nan

df['dp_clean'] = df['dp aktual'].apply(clean_currency)
df['cicilan_clean'] = df['cicilan'].apply(clean_currency)
df['otr_clean'] = pd.to_numeric(df['otr'], errors='coerce')
df['umur_clean'] = pd.to_numeric(df['umur'], errors='coerce')

# 3. Financial Ratio
df['dp_ratio'] = df['dp_clean'] / (df['otr_clean'] + 1)
df['cicilan_ratio'] = df['cicilan_clean'] / (df['otr_clean'] + 1)

print("Distribution Target:")
print(df['is_repeat'].value_counts(normalize=True) * 100)
```

✓ 0.8s

```
Distribution Target:
is_repeat
0    72.390602
1    27.609398
Name: proportion, dtype: float64
```

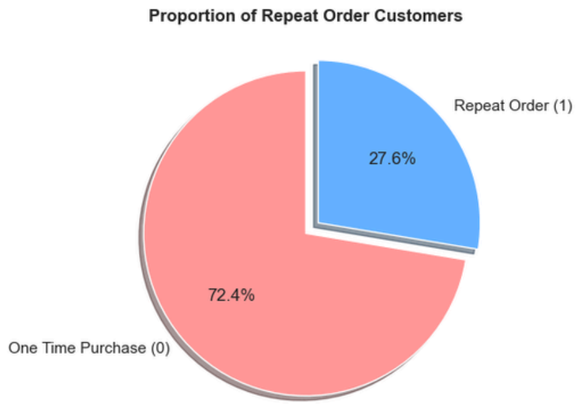
Missing Values Analysis

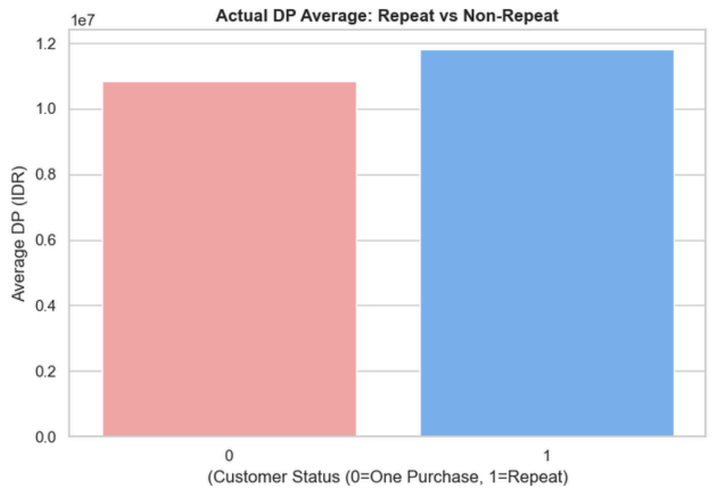
- High missing in date/dealer fieldstgl cetak (~100%) and dlr group (~80–90%) are mostly missing → consider dropping these features.
- Cleaned financial features are soliddp_clean, cicilan_clean, otr_clean, ratios, and transaction_count have ~0% missing → good after preprocessing.
- Demographic & categorical mostly complete Gender, occupation, dealer, location, color, age, etc. show very low/no missing → reliable for customer profiling.



Repeat Order Proportion & Average DP Comparison

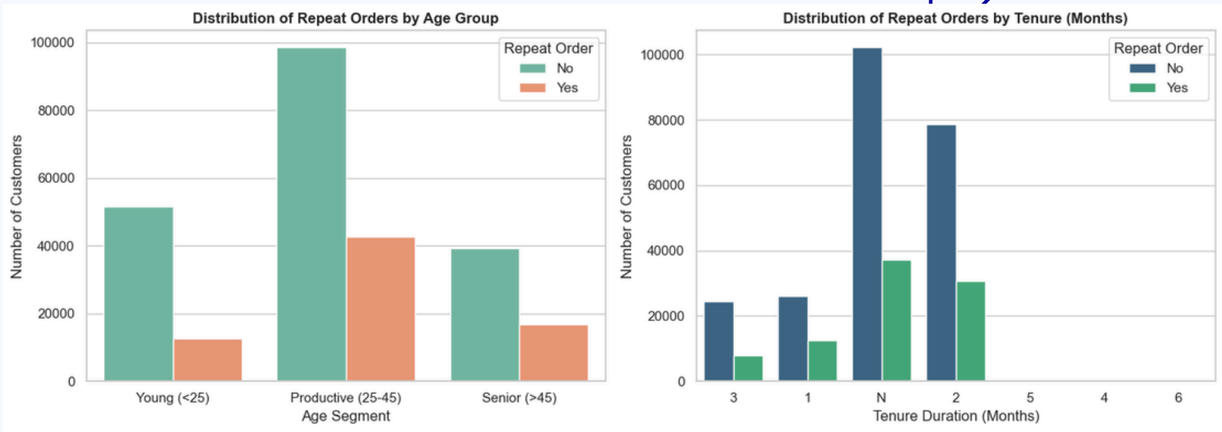
- Significant class imbalance 72.4% one-time buyers vs 27.6% repeat buyers → minority class requires careful handling (e.g. class weighting, F1 focus).
- Repeat buyers pay higher down payment Average actual DP is clearly higher for repeat customers (~11–12 million IDR) compared to one-time buyers (~10.5 million IDR).
- Strong early signal for loyalty Customers willing to pay larger upfront DP are more likely to return → valuable indicator for retention & targeting strategies.





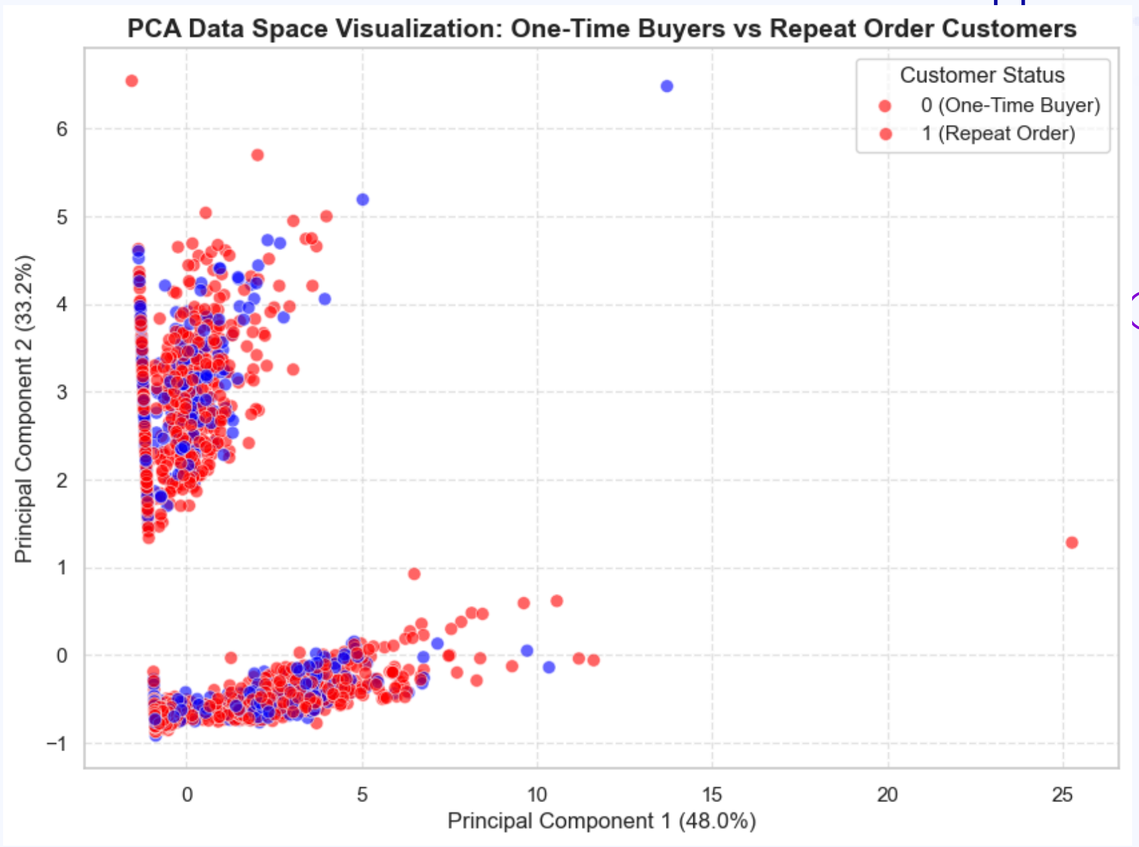
Distribution of Repeat Orders

- Productive age group dominates repeat orders The 25–45 age segment has the highest number of customers and the largest absolute repeat orders (orange bar tallest among groups).
- Mid-to-long tenure shows strongest repeat behavior Tenures labeled "N", "2", and "5" months have the highest repeat counts (green bars tallest), while very short tenures (1–3 months) show lower repeat rates.
- Clear targeting opportunity Customers aged 25–45 with medium-to-longer loan tenures represent the strongest repeat-buyer segment → ideal focus for loyalty programs, upselling, and retention efforts.



PCA Plot Insight

- Red and blue dots mix a lot → classes overlap, hard to separate with these features.
- Repeat buyers (blue) stay more in the main group → one-time buyers (red) have more outliers.
- PC1 shows big differences (48%), but still no clear line between repeat and non-repeat.



Preprocessing & Data Preparation

- Column Cleanup & Feature/Target Split Removed irrelevant/raw columns (e.g., IDs, dates, original unprocessed values) → separated features (X) from target is_repeat (y).
- Automatic Numeric & Categorical Detection + Missing Value Handling Detected numeric columns → imputed with median + scaled Detected categorical columns → imputed with 'Unknown' + one-hot encoded All handled inside a clean ColumnTransformer pipeline.
- Stratified Train-Test Split Split data 80:20 with stratify=y to preserve class balance (important due to ~28% repeat vs ~72% one-time buyers) → Data is now fully prepared and ready for modeling.

```python
# 1. Drop unnecessary columns
drop_cols = [
    'customer id', 'birth date', 'print date', 'assembly year',
    'dealer group', 'application date', 'actual dp', 'installment', 'otr', 'age', 'transaction_count']
df_model = df.drop(columns=[c for c in drop_cols if c in df.columns], errors='ignore')

# 2. Separate features (X) and target (y)
X = df_model.drop('is_repeat', axis=1)
y = df_model['is_repeat']

# 3. Numeric and categorical columns
numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = X.select_dtypes(include=['object']).columns.tolist()

# 4. Preprocessing pipeline and handling missing value
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())])
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))])
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
])

# 5. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=SEED,
    stratify=y)
```

Model Training & Evaluation with XGBoost

- Integrated Pipeline Preprocessing + XGBoost in one Pipeline → no leakage, consistent transforms.
- Imbalance Handling & CV Scale_pos_weight for 72:28 ratio + 3-fold Stratified CV → stable F1-score.
- Results & Visuals F1, ROC-AUC, classification report + Confusion Matrix, ROC, Precision-Recall plots → clear view of repeat buyer performance.

	precision	recall	f1-score	support
No Repeat (0)	0.80	0.66	0.72	46327
Repeat (1)	0.39	0.57	0.46	17669
...				
accuracy			0.63	63996
macro avg	0.60	0.62	0.59	63996
weighted avg	0.69	0.63	0.65	63996

```
high_cardinality_cols = []
for col in X.select_dtypes(include=['object']).columns:
    n_unique = X[col].nunique()
    if n_unique > 500:
        high_cardinality_cols.append(col)

if high_cardinality_cols:
    X = X.drop(columns=high_cardinality_cols, errors='ignore')

    numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
    categorical_features = X.select_dtypes(include=['object']).columns.tolist()

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
    ('onehot', OneHotEncoder(
        handle_unknown='ignore',
        sparse_output=False,
        max_categories=50,
        min_frequency=0.005
    ))
])
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
], remainder='drop')

model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', XGBClassifier(
        n_estimators=200,
        max_depth=6,
        learning_rate=0.05,
        tree_method='hist',
        random_state=42,
        scale_pos_weight=len(y_train[y_train==0]) / len(y_train[y_train==1]) if len(y_train[y_train==1]) > 0 else 1
    ))
])

cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)

try:
    cv_scores = cross_val_score(model_pipeline, X_train, y_train, cv=cv, scoring='f1')
    print(f"Average CV F1-Score: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
except Exception as e:
    print("Error during cross-validation:", str(e))
    print("Try running model.fit directly for further debugging.")
    raise

model_pipeline.fit(X_train, y_train)
y_pred = model_pipeline.predict(X_test)
y_prob = model_pipeline.predict_proba(X_test)[:, 1]
```

SHAP Analysis

- Age (umur) is the strongest driver Higher age strongly increases repeat probability (blue dots to the right), while younger customers tend to be one-time buyers.
- Financial ratios & OTR matter a lot Lower dp_ratio and higher otr_clean push predictions toward repeat orders → customers with higher vehicle value and lower down-payment proportion are more loyal.
- Segment, gender, dealer & occupation also influential MID segment, male gender (gender_N), certain dealers (kode dealer), and specific jobs/religion show positive impact → clear demographic & channel signals for targeting repeat buyers.